

Kanban CI/CD: The Steps

Every step that takes this project from a folder on a laptop to a site that redeploys itself on every merge — in order, with no explanations you don't need at the keyboard.

Companion to The Kanban Pipeline Manual

Time ~3 hours from scratch

Compiled 3 Sep 2026

What you are building, in five lines

1. You merge a pull request into `main`.
2. GitHub Actions rents a clean Linux machine, checks formatting and lint, and builds a Docker image.
3. It proves who it is to AWS with a short-lived token — no access keys anywhere — and pushes the image to ECR under two tags.
4. A second job opens SSH to your EC2 box for its own IP only, swaps the running container for the new image, and curls the site to prove it works.
5. If the site does not answer, it puts the previous image back and fails the run. Then it closes the SSH hole.

```
edit → commit → PR (format · lint · docker build) → merge  
→ build job (checks · OIDC · build · push :sha + :latest) → ECR  
→ deploy job (open :22 · ssh · pull · swap · health · close :22) → live
```

The eleven stages

- | | |
|-------------------------------|---------------------------------------|
| 01 Make the app shippable | 06 Launch and prepare EC2 |
| 02 Git & GitHub habits | 07 Add the GitHub variables & secrets |
| 03 Create the ECR repository | 08 Write the build job (CI) |
| 04 Trust GitHub in AWS (OIDC) | 09 Write the deploy job (CD) |
| 05 Create the IAM role | 10 First run, and what to check |
| | 11 Rolling back, and day-2 jobs |

BEFORE YOU START

An AWS account, a GitHub repository, Docker Desktop and the AWS CLI on your laptop, and a card on file with AWS (all of this fits in the free tier, but the account still needs one). Tags in the margin say where each stage happens: `LAPTOP` `GITHUB`

`AWS`

01 Make the app shippable `LAPTOP`

Nothing here is CI/CD. It is the part that makes CI/CD possible: an app that builds into one small, self-contained image.

STEP 1.1

Turn on standalone output

DO In `next.config.mjs`:

```
const nextConfig = { output: "standalone" };  
export default nextConfig;
```

Next then emits `.next/standalone` with its own `server.js` and only the modules it truly needs.

CHECK

`npm run build` creates `.next/standalone/server.js`. Without this the Dockerfile below cannot work.

STEP 1.2

Write the Dockerfile

DO

```
FROM node:22-alpine AS deps  
WORKDIR /app  
COPY package.json package-lock.json ./  
RUN --mount=type=cache,target=/root/.npm \  
    npm ci --fetch-timeout=600000 --fetch-retries=5 \  
        --fetch-retry-mintimeout=20000 \  
        --fetch-retry-maxtimeout=120000  
  
FROM node:22-alpine AS builder  
WORKDIR /app  
COPY --from=deps /app/node_modules ./node_modules  
COPY . .  
RUN npm run build  
  
FROM node:22-alpine AS runner  
WORKDIR /app  
ENV NODE_ENV=production  
COPY --from=builder /app/.next/standalone ./  
COPY --from=builder /app/.next/static ./next/static  
COPY --from=builder /app/public ./public  
EXPOSE 3000  
CMD ["node", "server.js"]
```

WHY

Three stages: install, build, ship. Only the third becomes the image, so no compiler and no dev dependencies travel to the server. The `--fetch-*` flags exist because npm timed out mid-build on a slow network.

STEP 1.3

Write `.dockerignore`

DO

```
node_modules
.next
.git
.env
.env.*
Dockerfile
.dockerignore
npm-debug.log
README.md
*.pem
```

WHY

`COPY . .` copies everything, including your SSH key and your `.env`, into a build layer. The `*.pem` line is not optional.

STEP 1.4

Give CI something to check

DO

Add to `package.json`:

```
"lint": "eslint",
"format": "prettier --write .",
"format:check": "prettier --check ."
```

Run `npm run format` once and commit the result, so the first CI run is not a wall of formatting failures.

CHECK

`docker build -t kanban-app .` succeeds locally, and `docker run -p 3000:3000 --env-file .env kanban-app` serves the app.

The pipeline is triggered by Git events, so your branching habit *is* the pipeline design.

STEP 2.1

Branch, pull request, merge — every time

DO

```
git checkout -b feat/thing
git add -A && git commit -m "feat: the thing"
git push -u origin feat/thing
gh pr create --base main --fill
```

WHY

Pushing a branch runs nothing — the workflow triggers on pull requests to `main` and on pushes to `main`. Opening the PR runs the checks; merging runs them again and then ships. **Test it on the PR, ship it on the merge.**

CHECK

Later, once the workflow exists: opening a PR shows a check running; merging shows a second, longer run.

WORTH DOING ONCE

Settings → Branches → add a rule for `main` requiring the CI check to pass. Until you do, a red check is only a suggestion.

03 Create the ECR repository AWS

The shelf your images sit on between being built and being run.

STEP 3.1

Create it

DO

ECR → Repositories → **Create** → Private → name `kanban-app` → leave the defaults.

Or:

```
aws ecr create-repository --repository-name kanban-app --region us-east-1
```

WRITE DOWN

The repository URI, e.g. `239068269281.dkr.ecr.us-east-1.amazonaws.com/kanban-app`. It contains your account ID, your region and the repo name — you need all three later.

CHECK

The repo is listed and empty. Keep the region consistent everywhere from here on.

This is what lets the pipeline into your account with no access key stored anywhere.

STEP 4.1

Add the identity provider

DO IAM → Identity providers → **Add provider** → OpenID Connect.

Provider URL `https://token.actions.githubusercontent.com`

Audience `sts.amazonaws.com`

WATCH

The URL field is pre-filled with `https://` — clear it before pasting, or you get `https://https://...`. There is no thumbprint step any more; AWS removed it.

STEP 4.2

Find out what GitHub will actually claim to be

DO If your repository has GitHub's **immutable identifiers** enabled — most new ones do — the subject carries numeric IDs, not names, and a policy written with names will never match. Get the real value from the API:

```
curl -s https://api.github.com/repos/<owner>/<repo> \
  | jq '{owner_id: .owner.id, repo_id: .id}'
```

The subject then looks like:

```
repo:Nasir1504@87933341/kanban-board@1345866624:ref:refs/heads/main
```

WHY NOW

Discovering this *after* the trust policy is written costs an afternoon, because AWS refuses the login without ever saying which part it disliked. Five minutes here, or five failed builds later.

05 Create the IAM role AWS

Two policies, and they answer different questions: **who** may become this role, and **what** may they do once they have.

STEP 5.1

The role and its trust policy — WHO

DO

IAM → Roles → Create role → **Web identity** → the provider you just added → audience `sts.amazonaws.com`. Name it `github-actions-ecr-push`. Then edit its trust policy to:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::<ACCOUNT>:oidc-provider/token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
      },
      "StringLike": {
        "token.actions.githubusercontent.com:sub":
          "repo:<owner>@<owner_id>/<repo>@<repo_id>:*"
      }
    }
  }]
}
```

WATCH

Leave the `sub` condition out and *any* GitHub repository in the world can assume your role. It is the whole security of this design.

The permissions policy — WHAT

DO Add an inline policy. Three statements: get a registry token, push to one repository, and open/close SSH for the deploy job.

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "GetAuthToken",
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*" },

    { "Sid": "PushToRepo",
      "Effect": "Allow",
      "Action": [
        "ecr:BatchCheckLayerAvailability",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload",
        "ecr:PutImage"
      ],
      "Resource": "arn:aws:ecr:<region>:<account>:repository/kanban-app" },

    { "Sid": "OpenAndCloseSsh",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeSecurityGroups",
        "ec2:AuthorizeSecurityGroupIngress",
        "ec2:RevokeSecurityGroupIngress"
      ],
      "Resource": "*" }
  ]
}
```

WATCH

Replace every `<placeholder>` — angle brackets are not a valid ARN and the editor will show a red cross. `GetAuthorizationToken` and the `Describe` calls genuinely require `"Resource": "*" ;` that is not laziness.

WRITE DOWN

The role ARN: `arn:aws:iam::<account>:role/github-actions-ecr-push`.

STEP 6.1

Launch the instance

DO EC2 → Launch instance → Ubuntu 24.04 LTS → **t3.micro** → create a new key pair (`.pem` , download it once) → allow HTTP and SSH → launch.

WATCH Not `t2.nano` — it runs out of memory and SSH starts closing on you. Give the root volume 16–20 GB; Docker images fill 8 GB quickly. Attach an **Elastic IP**, or the public IP changes on every stop/start and the deploy job stops finding the machine.

STEP 6.2

Security group

TYPE	PORT	SOURCE
SSH	22	My IP
HTTP	80	0.0.0.0/0
HTTPS	443	0.0.0.0/0

Never open 27017. The deploy job adds and removes its own port-22 rule; you do not add one for it.

CHECK Each saved rule shows an `sg-` ID. A rule with a blank source was never saved.

STEP 6.3

Connect, and install Docker

DO On Windows, fix the key's permissions first or SSH refuses it:

```
// PowerShell
icacls "path\to\key.pem" /inheritance:r
icacls "path\to\key.pem" /grant:r "$($env:USERNAME):(R)"

ssh -i "path\to\key.pem" ubuntu@<PUBLIC-IP>
```

Then, on the server:

```
sudo apt update && sudo apt upgrade -y
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker ubuntu
newgrp docker
```

CHECK `docker run --rm hello-world` works *without* `sudo` . Install Docker before `usermod` — the installer is what creates the group.

STEP 6.4

Capture the host key

DO

From your laptop:

```
ssh-keyscan -t ed25519 <PUBLIC-IP>
```

Keep the output line. It goes into a GitHub variable in the next stage so the runner cannot be fooled into deploying to an impostor.

07 Add the GitHub variables & secrets GITHUB

Repo → Settings → Secrets and variables → Actions. Two tabs; put each value on the right one.

NAME	TAB	VALUE
AWS_REGION	Variables	us-east-1
AWS_ROLE_ARN	Variables	the role ARN from stage 5
ECR_REPOSITORY	Variables	kanban-app
EC2_HOST	Variables	the instance's public IP
EC2_USER	Variables	ubuntu
EC2_HOST_KEY	Variables	the ssh-keyscan line from 6.4
APP_CONTAINER	Variables	kanban
APP_PORT	Variables	80
EC2_SSH_KEY	Secrets	the whole .pem file
MONGODB_URI	Secrets	your Mongo connection string

THREE WAYS THIS GOES WRONG

1. Copying a name out of a table brings a trailing space; GitHub rejects it with a message about alphanumeric characters. Type the names by hand.
2. The whole `NAME=value` pair gets pasted into the Name box. Name on top, value below.
3. The private key is pasted without its `-----BEGIN----- / -----END-----` lines, or loses its trailing newline. The error will say `invalid format` and mean "add a newline".

SECRET OR VARIABLE?

A secret is write-only and masked in logs; a variable is readable and printed. Your account ID, role ARN and server IP are not secrets — the firewall and the trust policy protect you, not obscurity. The SSH key and the database URI are secrets.

08 Write the build job (CI) GITHUB

One file: `.github/workflows/ci.yml`. This half checks the code and publishes the image.

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    permissions:
      id-token: write      # required for OIDC
      contents: read

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - run: npm ci
      - run: npm run format:check
      - run: npm run lint

      - name: Build Docker image (PR check)
        if: github.event_name == 'pull_request'
        run: docker build -t kanban-app .

      - name: Configure AWS credentials
        if: github.event_name == 'push'
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ vars.AWS_ROLE_ARN }
          aws-region: ${ vars.AWS_REGION }
          role-skip-session-tagging: true

      - name: Login to ECR
        id: ecr
        if: github.event_name == 'push'
        uses: aws-actions/amazon-ecr-login@v2

      - name: Build and push
        if: github.event_name == 'push'
        env:
          REGISTRY: ${ steps.ecr.outputs.registry }
          REPO: ${ vars.ECR_REPOSITORY }
        run: |
          docker build -t $REGISTRY/$REPO:$GITHUB_SHA \
            -t $REGISTRY/$REPO:latest .
          docker push $REGISTRY/$REPO:$GITHUB_SHA
          docker push $REGISTRY/$REPO:latest
```

LINE	WHY IT IS THERE
<code>id-token: write</code>	Lets the job request an OIDC token. Missing it gives the useless error "Credentials could not be loaded".
<code>if: ... = 'pull_request'</code>	PRs build the image but never publish it. Unreviewed code must not overwrite <code>:latest</code> .
<code>if: ... = 'push'</code>	Only a merge to <code>main</code> touches AWS.
<code>role-skip-session-tagging</code>	Session tags need an extra permission in the trust policy. Simpler to switch them off.
<code>id: ecr</code>	Makes <code>steps.ecr.outputs.registry</code> available, so the account ID is never typed into the YAML.
two tags	<code>:\$GITHUB_SHA</code> is the immutable identity that makes rollback possible; <code>:latest</code> is a convenience pointer.

09 Write the deploy job (CD) GITHUB

The second half of the same file. Four moves: open the door, swap the container, prove it works, close the door.

a · Open the door

```
deploy:
  if: github.event_name == 'push'
  needs: build
  runs-on: ubuntu-latest
  permissions:
    id-token: write
    contents: read

  steps:
    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: ${ vars.AWS_ROLE_ARN }
        aws-region: ${ vars.AWS_REGION }
        role-skip-session-tagging: true

    - name: Login to ECR
      id: ecr
      uses: aws-actions/amazon-ecr-login@v2

    - name: Open SSH for this runner
      id: sg
      env:
        EC2_HOST: ${ vars.EC2_HOST }
      run: |
        set -e
        pipefail
        RUNNER_IP="$(curl -fsS --retry 3 https://checkip.amazonaws.com)"
        SG_ID="$(aws ec2 describe-instances \
          --filters "Name=ip-address,Values=$EC2_HOST" \
          --query 'Reservations[0].Instances[0].SecurityGroups[0].GroupId' \
          --output text)"

        echo "sg_id=$SG_ID" >> "$GITHUB_OUTPUT"
        echo "cidr=$RUNNER_IP/32" >> "$GITHUB_OUTPUT"

        aws ec2 authorize-security-group-ingress --group-id "$SG_ID" \
          --ip-permissions "IpProtocol=tcp,FromPort=22,ToPort=22,\
IpRanges=[{CidrIp=$RUNNER_IP/32,\
Description=gha-run-$GITHUB_RUN_ID}]" >/dev/null
```

A GitHub runner has a different IP on every run, so there is nothing to allowlist in advance. The job asks AWS what its own address is, opens a one-address hole, and records what it opened so the closing step can find it even if the deploy dies.

b · Connect and hand over the secrets

```
- name: Deploy to EC2
  env:
    REGISTRY: ${ steps.ecr.outputs.registry }
    REPO: ${ vars.ECR_REPOSITORY }
    AWS_REGION: ${ vars.AWS_REGION }
    EC2_HOST: ${ vars.EC2_HOST }
    EC2_USER: ${ vars.EC2_USER }
    EC2_HOST_KEY: ${ vars.EC2_HOST_KEY }
    CONTAINER: ${ vars.APP_CONTAINER }
    APP_PORT: ${ vars.APP_PORT }
    SSH_KEY: ${ secrets.EC2_SSH_KEY }
    MONGODB_URI: ${ secrets.MONGODB_URI }
  run: |
    set -euo pipefail

    mkdir -p ~/.ssh
    printf '%s\n' "$SSH_KEY" > ~/.ssh/deploy_key
    chmod 600 ~/.ssh/deploy_key
    printf '%s\n' "$EC2_HOST_KEY" > ~/.ssh/known_hosts

    IMAGE="$REGISTRY/$REPO:$GITHUB_SHA"
    ECR_PASSWORD="$(aws ecr get-login-password --region "$AWS_REGION")"
    echo "::add-mask::$ECR_PASSWORD"

    ssh -i ~/.ssh/deploy_key -o StrictHostKeyChecking=yes \
      "$EC2_USER@$EC2_HOST" 'bash -s' <<EOF
```

`printf` not `echo`, because the key needs its trailing newline. `bash -s` with a heredoc, because a script on **stdin** is invisible to `ps` on the server while a command line is not — and this script has secrets interpolated into it.

c • Swap, verify, roll back — this part runs on the server

```
set -euo pipefail
APP_DIR="\$HOME/kanban"
mkdir -p "\$APP_DIR"

umask 077
cat > "\$APP_DIR/.env" <<'ENVEOF'
MONGODB_URI=$MONGODB_URI
ENVEOF

printf '%s' '$ECR_PASSWORD' \
  | docker login --username AWS --password-stdin '$REGISTRY'
docker pull '$IMAGE'

PREVIOUS="\$(docker inspect --format '{{.Image}}' \
  '$CONTAINER' 2>/dev/null || true)"

start() {
  docker rm -f '$CONTAINER' >/dev/null 2>&1 || true
  docker run -d --name '$CONTAINER' --restart unless-stopped \
    -p '$APP_PORT':3000 --env-file "\$APP_DIR/.env" "\$1" >/dev/null
}

healthy() {
  for _ in \$(seq 1 30); do
    curl -fsS -o /dev/null "http://localhost:$APP_PORT/" && return 0
    sleep 2
  done
  return 1
}

start '$IMAGE'

if ! healthy; then
  echo "health check failed for $IMAGE"
  docker logs --tail 50 '$CONTAINER' || true
  if [ -n "\$PREVIOUS" ]; then
    echo "rolling back to \$PREVIOUS"
    start "\$PREVIOUS"
  fi
  exit 1
fi

docker image prune -f >/dev/null
echo "deployed $IMAGE"
EOF
```

THE ONE RULE FOR READING THIS BLOCK

A bare `$VAR` is filled in by GitHub before the script is sent. A `\$VAR` is evaluated on the server. Get that backwards and you get empty values, not errors.

d • Close the door, whatever happened

```
- name: Close SSH
  if: always() && steps.sg.outputs.sg_id != ''
  run: |
    aws ec2 revoke-security-group-ingress \
      --group-id '${ steps.sg.outputs.sg_id }' \
      --protocol tcp --port 22 \
      --cidr '${ steps.sg.outputs.cidr }' >/dev/null
```

`always()` is the point of this step: without it, a failed deploy leaves your firewall open exactly when you are least likely to be watching.

10 First run, and what to check

STEP 10.1

Ship it

DO

Commit the workflow on a branch, open a PR, watch the checks, merge.

CHECK, IN ORDER

- ☐ PR run is green and includes a `docker build` , but no AWS steps.
- ☐ Merge run's `build` job ends with two `docker push` lines.
- ☐ ECR shows **one image with two tags**: the commit SHA and `latest` .
- ☐ `deploy` job's log ends with `deployed .../kanban-app:<sha>` .
- ☐ `http://<PUBLIC-IP>` serves the new build. Use **http** — there is no certificate.
- ☐ The security group has no `gha-run-...` rule left in it.

If a step goes red

MESSAGE	WHAT IT REALLY MEANS	FIX
<code>Not authorized to perform sts:AssumeRoleWithWebIdentity</code>	The trust policy's <code>sub</code> does not match what GitHub sent. AWS will not say which part.	Print the real claims (below) and paste the exact <code>sub</code> into the policy.
<code>Credentials could not be loaded</code>	No OIDC token was requested.	Add <code>permissions: id-token: write</code> to the job.
<code>port 22: Connection timed out</code>	The runner is not the IP you allowlisted, and never will be.	The Open/Close SSH steps in stage 9.
<code>UnauthorizedOperation</code> on an <code>ec2:</code> call	The role may push images and nothing else.	Add exactly the action named in the error.
<code>invalid format</code> reading the key	The secret lost its header, footer or trailing newline.	Re-paste the whole <code>.pem</code> ; keep <code>printf '%s\n'</code> .
health check failed, rolled back	The image starts but does not serve.	Read the 50 log lines the job printed. Usually a missing env var or a crash on boot.
CI never ran at all	You pushed a branch; the triggers are <code>main</code> and PRs to <code>main</code> .	Open the pull request.

THE DEBUG STEP WORTH KEEPING

When AWS refuses the login and every value looks right, stop checking the console and read what GitHub actually sent:

```
- name: Debug OIDC claims
  run: |
    TOKEN=$(curl -sS -H "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_TOKEN" \
      "$ACTIONS_ID_TOKEN_REQUEST_URL&audience=sts.amazonaws.com" | jq -r '.value')
    PAYLOAD=$(echo "$TOKEN" | cut -d. -f2)
    PAYLOAD="${PAYLOAD}${printf '%*s' $(( (4 - ${#PAYLOAD} % 4) % 4 )) ' ' | tr ' ' '=')}"
    echo "$PAYLOAD" | tr '_-' '/+' | base64 -d | jq '{iss, aud, sub}'
```

It prints no credentials, runs in a second, and turns a whole afternoon into one line of output. Leave it in the file, commented out.

11 Rolling back, and day-2 jobs

Roll back

SITUATION	DO THIS
The deploy failed its health check	Nothing — the job already restarted the previous image and went red.
It deployed fine but the change is wrong	Actions → the last good run → Re-run jobs . It redeploys that run's SHA.
GitHub is down, or you need it now	SSH in and start the old image by hand: <pre>docker rm -f kanban docker run -d --name kanban --restart unless-stopped \ -p 80:3000 --env-file ~/kanban/.env \ <REGISTRY>/kanban-app:<old-sha></pre>
The bad code is on <code>main</code>	<code>git revert</code> the merge and push. The pipeline ships the revert like any other change.

Checks worth running now and then

```
# which image is actually running?
docker inspect --format '{{.Config.Image}}' kanban

# disk, before it fills
df -h ; docker system df

# the app's own output
docker logs -f kanban

# any firewall rule left behind by a failed deploy
aws ec2 describe-security-groups --group-ids <SG_ID> \
  --query 'SecurityGroups[0].IpPermissions'
```

The next four improvements, in order

1. **Tests.** Add `npm test` as a gate before the build. Format and lint check style, not behaviour — this is the biggest hole in the pipeline.
2. **An ECR lifecycle policy**, so old images expire instead of accumulating storage cost.
3. **HTTPS.** A domain, a reverse proxy, a Let's Encrypt certificate.
4. **Zero downtime.** Today the container stops before the new one starts — a few seconds of 502. Start the new one on a second port, health-check it, then switch.

★ The whole thing on one page

The four questions every pipeline answers

1 • When?	push to <code>main</code> , or a PR targeting <code>main</code>
2 • What gates?	prettier → eslint → docker build → a live health check
3 • What artifact?	one Docker image, tagged <code>:<sha></code> and <code>:latest</code> , stored in ECR
4 • How does it ship, and how do I undo it?	the <code>deploy</code> job swaps the container to the <code>:<sha></code> image; it rolls itself back on a failed health check, and you roll back by re-running an older green run

Where every piece lives

PIECE	HOME
The recipe	<code>Dockerfile</code> in the repo
The pipeline	<code>.github/workflows/ci.yml</code>
The identity	IAM role <code>github-actions-ecr-push</code> , assumed by OIDC — no stored AWS key
The artifact	ECR repository <code>kanban-app</code>
The runtime	one container on one EC2 instance, port 80 → 3000
The config	GitHub secrets, rewritten into <code>~/kanban/.env</code> on every deploy

Five rules that prevent most of the pain

1. **Never build on the server.** Build in CI; the server only pulls and runs.
2. **Never bake a secret into an image.** Runtime `--env-file`, always.
3. **Always deploy the SHA tag, never `latest`.** It is your rollback and your audit trail.
4. **Give every identity the smallest permission that works** — and expect to add one each time the pipeline learns a new trick.
5. **When one side will not say what it received, print what the other side sent.**

A pipeline is not a trusted insider

It is an anonymous machine, rented for ninety seconds, arriving with no network position and no permissions except the ones you wrote down. Almost every "but it works from my laptop" turns out to be one of those two sentences.